Localization and Object Detection

From one label to a variable set of boxes

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

What task are we solving?

Where we are

So far a network mapped an image to **one** answer:

$$f_\theta(x) \to \hat{y}, \quad \hat{y} = \mathrm{softmax}(Wh) \in \Delta^{K-1}$$

Classification asks **what is in the image?** — a single label. But most real images contain **several** things, each **somewhere**. This lecture is about **what** and **where**, for a variable number of objects.

The spine of this lecture

Classification predicts **one label**.
Detection predicts a **variable set of objects**,
each with a **class** and a **box**.

the output is no longer a fixed-size vector — it is a **set** whose size depends on the image.

Classification: $x \to y$

One image in, one categorical label out.

$$f_\theta(x) = \hat{y} \in \{1, ..., K\}$$

- output shape is **fixed**: a K-way distribution;
- no notion of **location** — the object could be anywhere;
- the whole image is assumed to be **about one thing**.

Localization: $x \to (y, b)$

Now also report **where** the (single, dominant) object is:

$$f_\theta(x) = \left(\hat{y}, \hat{b}\right), \quad \hat{b} \in \mathbb{R}^4$$

- $\hat{y}$ — the class, as before;
- $\hat{b}$ — **one** bounding box (four numbers);
- still **exactly one** object per image — the output is fixed-size.

classification + a box regression head

Detection: $x \to \{(y_i, b_i, s_i)\}_{i=1}^{N}$

Report **every** object: a **variable-length set**.

$$f_\theta(x) = \left\{ \left(\hat{y}_i, \hat{b}_i, s_i \right) \right\}_{i=1}^{N}$$

- $\hat{y}_i$ — class of object i; $\hat{b}_i$ — its box; s_i — a confidence score;
- N **varies per image** — 0 objects, 1, or dozens;
- this variable N is what makes detection **fundamentally harder**.

One more task — but that is next time

Segmentation pushes further: a class label for **every pixel**, not just a box. That is **Lecture 10**. Today we stop at boxes.

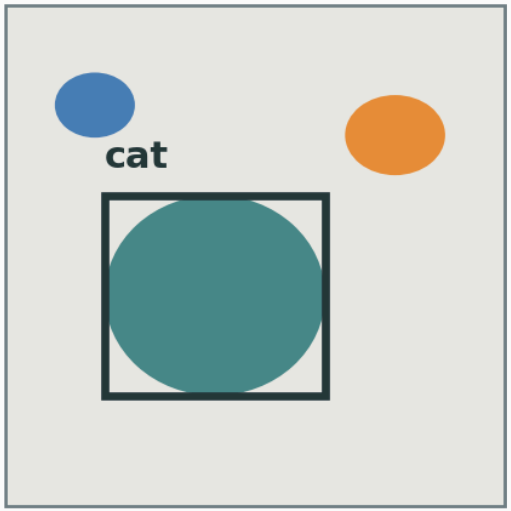
Task	Output	Granularity
classification	$\hat{y}$	whole image
localization	$\left(\hat{y}, \hat{b}\right)$	one object
detection	$\left\{\left(\hat{y}_i, \hat{b}_i, s_i\right)\right\}$	many objects
segmentation	pixel labels	every pixel (L10)

The same image, three tasks

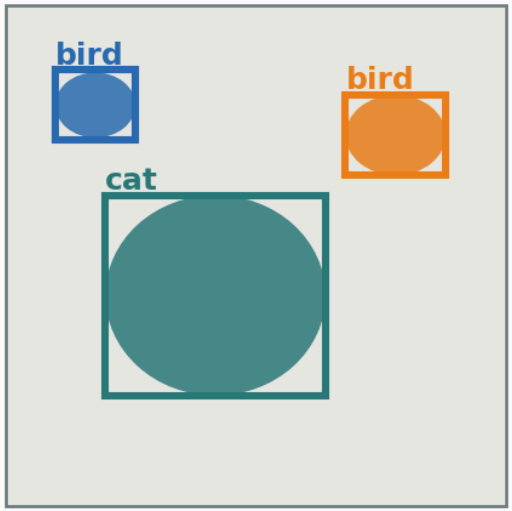
V

classification
$x \to y$

localization
$x \to (y, b)$



detection
$x \to \{(y_i, b_i)\}$



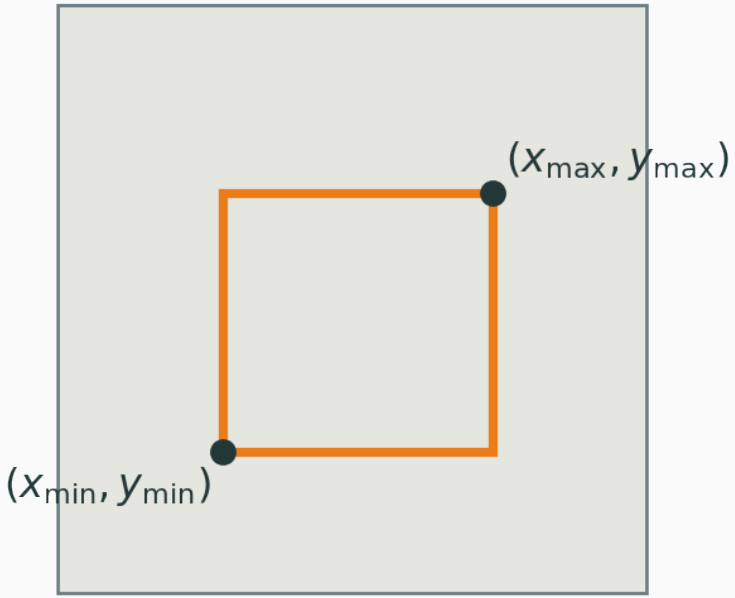
one label · one box · a **set** of boxes — the output grows richer left to right.

Bounding boxes and localization

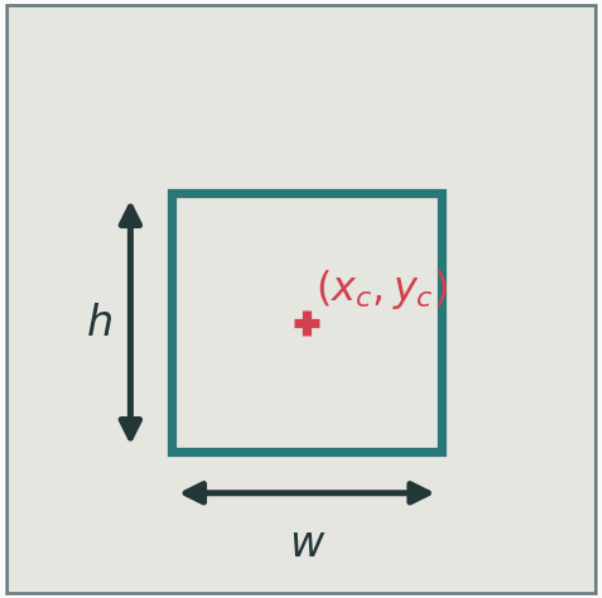
What is a bounding box?

A box is **four numbers** — two equivalent parameterizations:

corner form



center + size form



$$b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \quad \text{or} \quad b = (x_c, y_c, w, h)$$

Normalize the coordinates

Raw pixel coordinates depend on image size. **Divide out** width W and height H:

$$\tilde{x} = \frac{x}{W}, \quad \tilde{y} = \frac{y}{H}, \quad \tilde{w} = \frac{w}{W}, \quad \tilde{h} = \frac{h}{H}$$

Normalized coordinates live in $[0, 1]$ regardless of resolution — so the same network handles images of **any** size, and the regression targets are **well-scaled**.

A localization network

Reuse a classification backbone; add a **second head**.

$$h = f_\theta(x) \qquad \text{(shared backbone features)}$$

$$\hat{p} = \mathrm{softmax}(W_c\, h) \qquad \text{(classification head)}$$

$$\hat{b} = W_b\, h \qquad \text{(box regression head)}$$

one backbone, two heads — a class distribution and four box numbers

Training with two objectives

The network must be **right about the class** and **tight around the object**:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \, \mathcal{L}_{\text{box}}$$

- $\mathcal{L}_{\text{cls}}$ — cross-entropy on $\hat{p}$ (Lecture 1);
- $\mathcal{L}_{\text{box}}$ — a **regression** loss on $\hat{b}$;
- λ — a weight balancing the two (they have different units/scales).

This **multi-task loss** — a shared trunk with several weighted heads — recurs throughout detection.

The box regression loss

Penalize the difference between predicted and true box coordinates:

$$\mathcal{L}_{\text{box}} = \left\| \hat{b} - b \right\|_1 = \sum_{j=1}^{4} |\hat{b}_j - b_j| \qquad (L_1)$$

or the squared version:

$$\mathcal{L}_{\text{box}} = \left\| \hat{b} - b \right\|_2^2 = \sum_{j=1}^{4} \left(\hat{b}_j - b_j \right)^2 \qquad (L_2)$$

L_2 punishes large errors hard (sensitive to outliers); L_1 is steadier but kinks at 0.

Smooth L_1: the best of both

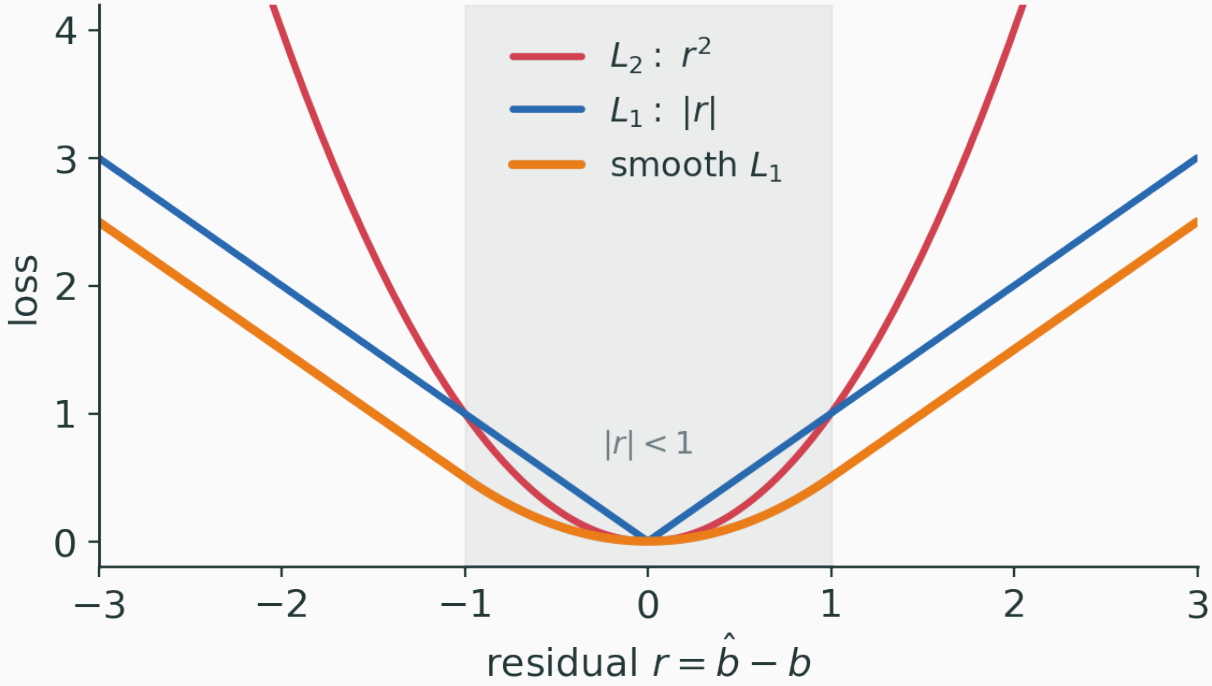
D

Quadratic near 0 (stable gradient), linear far out (robust to outliers):

$$\text{smooth}_{L_1}(r) = \begin{cases} \frac{1}{2}r^2 & \text{if } |r| < 1 \\ |r| - \frac{1}{2} & \text{otherwise} \end{cases}$$

$r = \hat{b}_j - b_j$ is the per-coordinate residual — the pieces meet smoothly at $|r| = 1$.

Smooth L_1 vs L_1 vs L_2



near 0 it curves like L_2 (no kink) · far out it grows like L_1 (no outlier blow-up).

Worked example: localization loss

D

True box $b = (0.5, 0.5, 0.4, 0.2)$, prediction $\hat{b} = (0.6, 0.45, 0.5, 0.3)$ (normalized). Per-coordinate absolute residuals:

$$|0.6 - 0.5| = 0.10, \quad |0.45 - 0.5| = 0.05$$

$$|0.5 - 0.4| = 0.10, \quad |0.3 - 0.2| = 0.10$$

$$\mathcal{L}_{\text{box}} = 0.10 + 0.05 + 0.10 + 0.10 = 0.35$$

$$\boxed{L_1 = 0.35}$$

Interactive: box-regression loss I

INTERACTIVE

(to build) — drag the predicted box around a fixed ground-truth box and watch L_1, L_2, and smooth-L_1 update coordinate-by-coordinate, and see where each loss's gradient is large or flat.

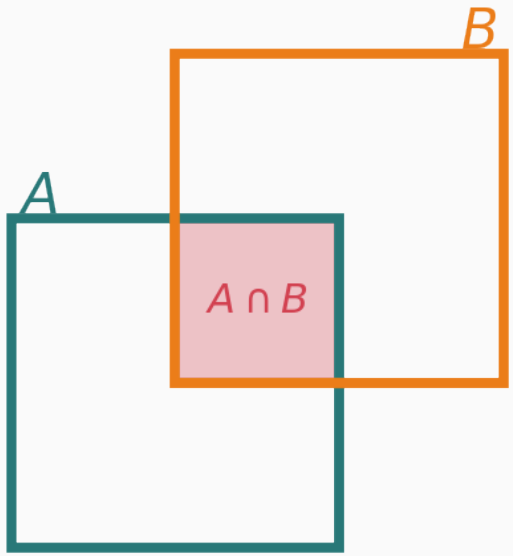
Q. Coordinate losses treat all four numbers independently. What about the box do they **not** directly measure?

IoU and detection metrics

Intersection over Union

The standard measure of **box overlap** — area of overlap ÷ area of union:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]$$



$|A| = 100, \quad |B| = 100, \quad |A \cap B| = 25$

$|A \cup B| = 175, \quad \text{IoU} = 25/175 \approx 0.143$

Worked example: IoU

D

Two boxes, each of area 100; their overlap has area 25. Union by inclusion–exclusion:

$$|A \cup B| = |A| + |B| - |A \cap B| = 100 + 100 - 25 = 175$$

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{25}{175} \approx 0.143$$

IoU $= 25/175 \approx 0.143$

Why coordinate loss is not enough

Smooth-L_1 drives the four numbers together — but that is **not the same** as overlap.

- two boxes can have **equal** coordinate error yet **very different** IoU;
- IoU is **scale-invariant**; a fixed pixel error matters more for a **small** box;
- the metric we **report** (IoU) differs from the loss we **optimize** (coordinates).

so: optimize an IoU-based loss directly

The trouble with plain IoU loss

Use $\mathcal{L}_{\text{IoU}} = 1 - \text{IoU}$? It works **when boxes overlap** — but:

If the boxes **do not overlap**, $\text{IoU} = 0$ for **all** non-overlapping configurations. The loss is flat — **zero gradient** — so it cannot pull a far-off box toward the target.

GIoU: a gradient even with no overlap

optional

Generalized IoU adds a term using the smallest box C enclosing both:

$$\text{GIoU} = \text{IoU} - \frac{|C \setminus (A \cup B)|}{|C|} \in [-1, 1]$$

- when A, B overlap, GIoU $\approx$ IoU;
- when they are **apart**, the enclosing-box term **still varies with distance** — a usable gradient.

later variants (DIoU, CIoU) refine this further — all share the same fix: keep a signal when IoU is 0.

Precision and recall

To **score a detector**, first count matches (a prediction is a **hit** if IoU with a true box $> \tau$):

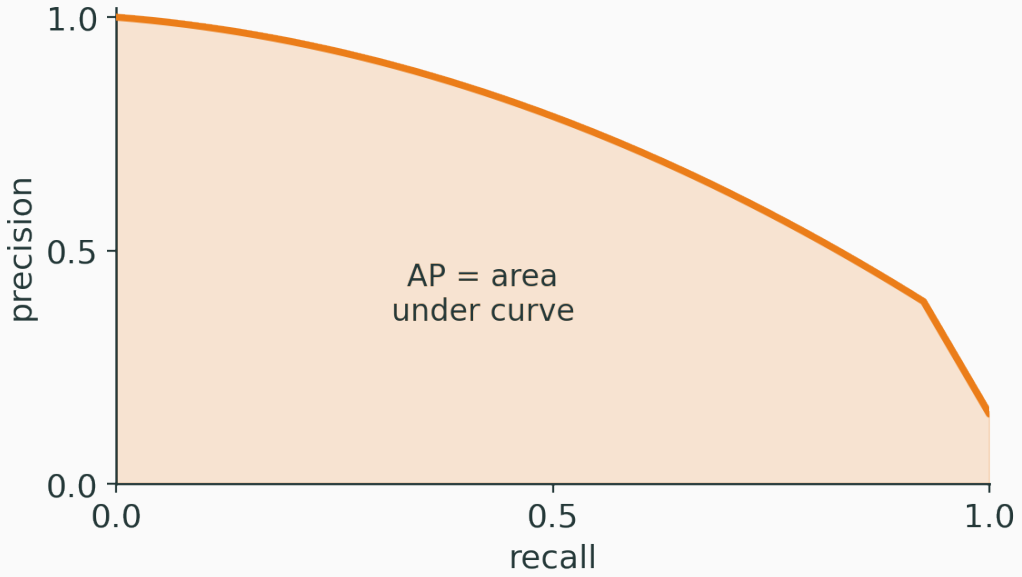
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **precision** — of the boxes I predicted, how many were **right**?
- **recall** — of the objects that exist, how many did I **find**?

Lowering the score threshold finds more objects (↑ recall) but admits more false alarms (↓ precision). There is a **tradeoff curve**.

Average Precision (AP)

Sweep the score threshold; plot precision vs recall; AP is the **area under it**:



$$\text{AP} = \int_0^1 \text{precision}(r)\, dr \quad \in [0, 1]$$

mean Average Precision (mAP)

AP is **per class** at a **given** IoU threshold. Average to summarize a detector:

$$\text{mAP} = \frac{1}{K} \sum_{k=1}^{K} \text{AP}_k$$

- average over all **classes** k (car, person, dog, ...);
- modern benchmarks **also** average over IoU thresholds $\tau \in \{0.5, 0.55, ..., 0.95\}$;
- reported as e.g. **mAP@0.5** or **mAP@[.5:.95]**.

mAP — one number summarizing detection quality

Interactive: IoU and mAP

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/object-detection

Drag two boxes to see IoU update live; then move a score threshold along a PR curve and watch precision, recall, and the AP area respond.

Q. A detector reports every object with **very low** confidence. What happens to its precision, and to its recall?

Building a detector

Why detection is hard

Unlike classification, the output is a **set of unknown size**:

- **how many** objects? unknown, and it varies per image;
- **where / what scale**? anywhere, any size;
- **class imbalance** — the vast majority of image regions are **background**;
- **duplicates** — one object easily triggers **many** overlapping predictions.

Every design choice below is a response to one of these four problems.

The core idea: dense candidates

Rather than guess N, **predict at many fixed locations and scales**, then filter.

- tile the image with a **dense grid** of candidate regions;
- at each, ask: **is there an object here? what class? what exact box?**
- most candidates say "background"; a few fire — then we **clean up duplicates**.

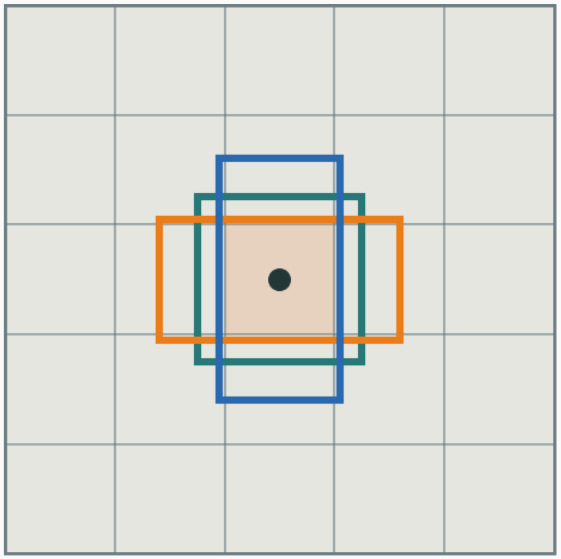
turn a variable-size set problem into a fixed, dense prediction problem

Anchor boxes

Predefined **reference boxes** of assorted scales/aspect-ratios at every location:

$$a = (x_a, y_a, w_a, h_a)$$

anchors: fixed reference boxes at every cell



the network **adjusts** an anchor rather than inventing a box from scratch.

Predict box offsets, not raw coordinates

The network outputs **corrections** to each anchor — easier to learn and well-scaled:

$$t_x = \frac{x - x_a}{w_a}, \qquad t_y = \frac{y - y_a}{h_a}$$

$$t_w = \log\left(\frac{w}{w_a}\right), \qquad t_h = \log\left(\frac{h}{h_a}\right)$$

Offsets are **relative** to the anchor: centers in anchor-widths, sizes in log-scale. A prediction of **all zeros** reproduces the anchor exactly.

What the detection head outputs

Per anchor (at each grid location), the head emits:

- **objectness** $\hat{o} \in [0, 1]$ — is there an object here at all?
- **class logits** — a K-way distribution **if** there is;
- **box offsets** (t_x, t_y, t_w, t_h) — how to reshape the anchor.

for A anchors and K classes: $A \times (1 + K + 4)$ numbers per location.

The detection loss

Sum the three jobs, over **positive** (object) and relevant anchors:

$$\mathcal{L} = \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{box}}$$

- $\mathcal{L}_{\text{obj}}$ — objectness (object vs background), over **all** anchors;
- $\mathcal{L}_{\text{cls}}$ — class cross-entropy, on **positive** anchors only;
- $\mathcal{L}_{\text{box}}$ — smooth-L_1 on offsets, on **positive** anchors only.

Which anchors are positive?

D

Assign each anchor a label by its IoU with the **ground-truth** boxes:

$$\mathrm{IoU}\big(a, b_{\mathrm{gt}}\big) > \tau_+ \implies \text{positive (an object)}$$

$$\mathrm{IoU}\big(a, b_{\mathrm{gt}}\big) < \tau_- \implies \text{negative (background)}$$

- anchors in between ($\tau_- \le \mathrm{IoU} \le \tau_+$) are **ignored** (ambiguous);
- a positive anchor's box/class targets come from the matched b_{gt};
- typical thresholds: $\tau_+ \approx 0.7$, $\tau_- \approx 0.3$.

The class-imbalance problem

With thousands of anchors per image, **almost all** are background.

A sea of easy negatives **drowns out** the gradient from the few real objects. Plain cross-entropy, summed over all anchors, is dominated by easy background.

foreground : background can be **1 : 1000s** — the loss needs to **ignore the easy majority**.

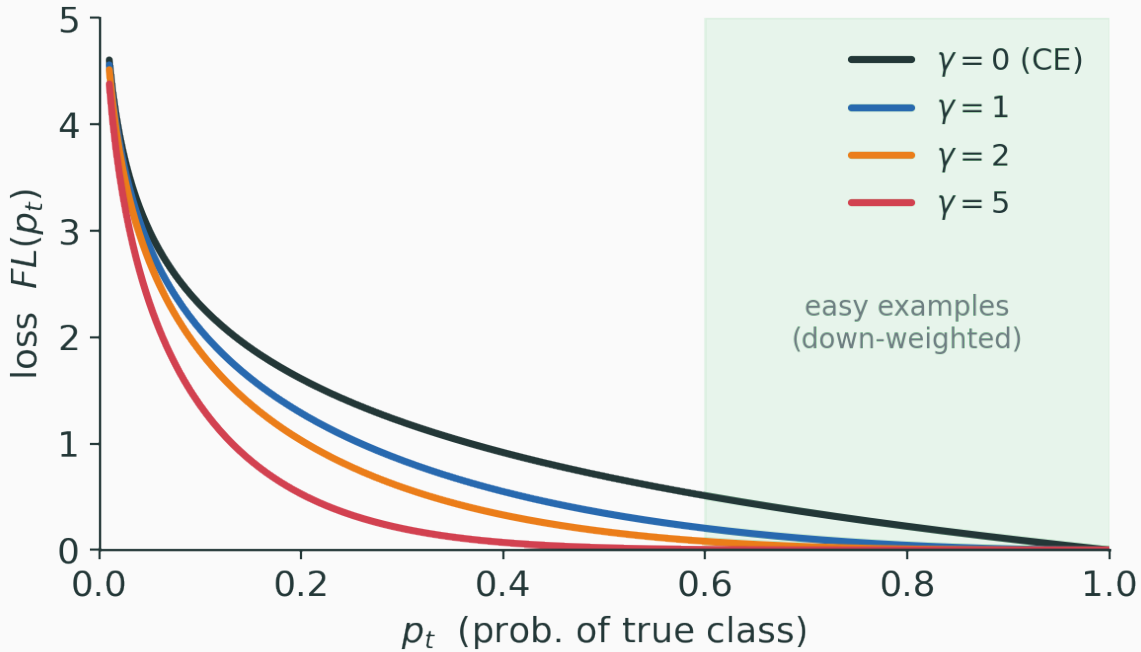
Down-weight **easy, well-classified** examples by a factor $(1-p_t)^\gamma$:

$$\mathrm{FL}(p_t) = -(1-p_t)^\gamma \log p_t$$

where p_t is the predicted probability of the **true** class.

- $\gamma = 0$ recovers **ordinary cross-entropy**;
- large p_t (easy) $\Rightarrow (1-p_t)^\gamma \approx 0 \Rightarrow$ **tiny** loss;
- small p_t (hard) $\Rightarrow$ factor $\approx 1 \Rightarrow$ loss **kept**.

Focal loss down-weights easy examples



higher $\gamma \Rightarrow$ easy examples ($p_t \to 1$) contribute almost nothing — built for **dense one-stage** detectors.

Interactive: anchor assignment

I

INTERACTIVE

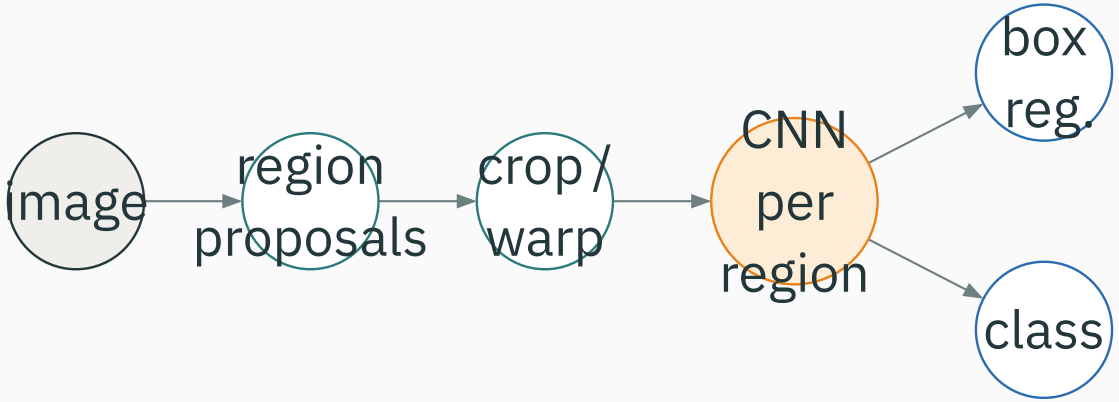
(to build) — drop a ground-truth box on an anchor grid and watch anchors light up **positive / ignored / negative** as you sweep τ_+ and τ_-.

Q. If you set τ_+ very high (say 0.9), what happens to the number of positive anchors — and to training?

Two-stage detectors: the R-CNN family

R-CNN: propose, then classify

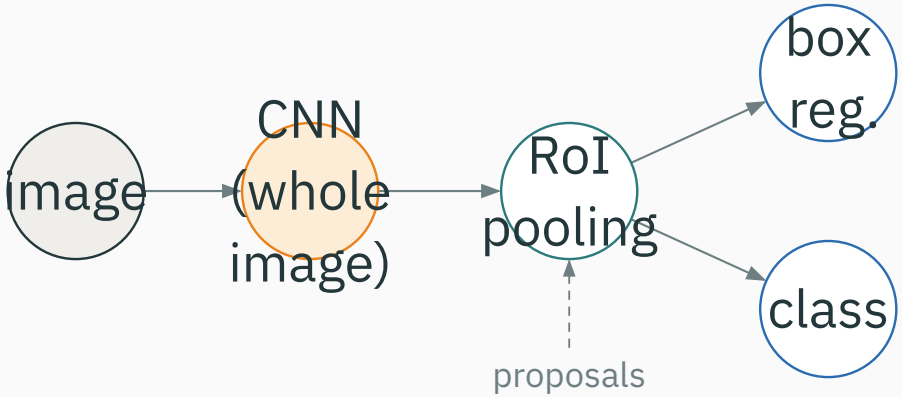
Stage 1 proposes regions (classic **selective search**); stage 2 runs a CNN on **each**.



A CNN forward pass **per region** (thousands of crops) — **slow**, and the multi-stage training (proposals, CNN, SVMs, regressors) is cumbersome.

Fast R-CNN: share the convolution

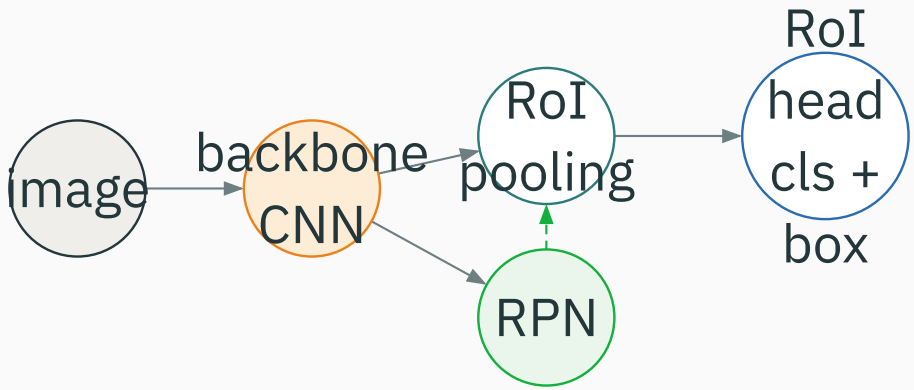
Run the CNN **once** over the whole image; pool features **per proposal** (**RoI pooling**).



The expensive convolutions are computed **once** and **shared** across all proposals — a large speedup, and the heads train **end-to-end**.

Faster R-CNN: learn the proposals too

Replace hand-crafted proposals with a **Region Proposal Network** (RPN) on the same features.



the RPN and the detection head **share the backbone** — proposals become **neural**, fast, and trainable.

The two losses of Faster R-CNN

Both stages carry an **objectness/class** term and a **box** term:

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{obj}} + \lambda\,\mathcal{L}_{\text{box}} \qquad \text{(is it an object? rough box)}$$

$$\mathcal{L}_{\text{RoI}} = \mathcal{L}_{\text{cls}} + \lambda\,\mathcal{L}_{\text{box}} \qquad \text{(which class? refined box)}$$

trained jointly — one network, two stages, four loss terms

The two-stage intuition

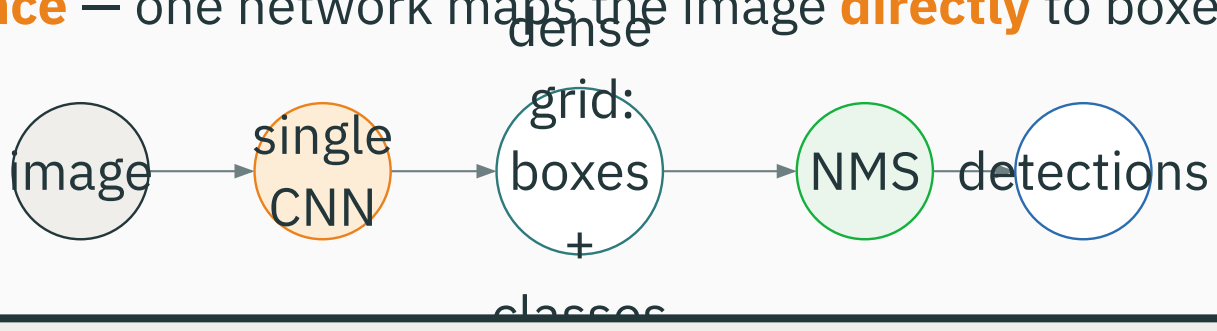
Stage	Question	Output
1 — RPN	where **might** objects be?	class-agnostic proposals + rough boxes
2 — RoI head	**what** is it, exactly **where**?	class + refined box per proposal

Stage 1 casts a wide net (high recall, coarse); stage 2 is precise (class + tight box). **Historically** this two-stage split gave strong localization.

One-stage detectors and NMS

YOLO: detection in a single pass

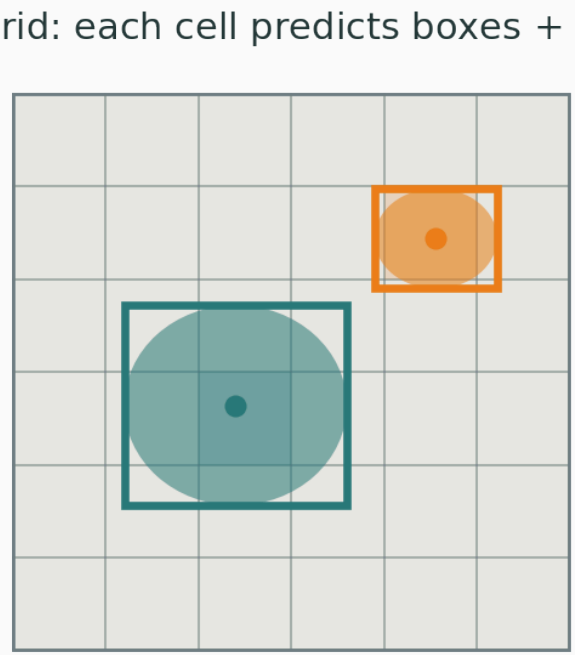
You Only Look Once — one network maps the image **directly** to boxes + classes.



No separate proposal stage: a **single** forward pass regresses all boxes and classes at once, then NMS removes duplicates. Simple and **fast**.

The grid intuition

Divide the image into an $S \times S$ grid; each cell predicts boxes + classes:



the cell containing an object's **center** is responsible for predicting it.

The one-stage loss

The familiar three terms, over the **dense grid** (no proposal stage):

$$\mathcal{L} = \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{cls}}$$

- $\mathcal{L}_{\text{box}}$ — box offset regression (smooth-L_1 / IoU-based);
- $\mathcal{L}_{\text{obj}}$ — objectness / confidence per cell-anchor;
- $\mathcal{L}_{\text{cls}}$ — class prediction for cells that contain an object.

same ingredients as Faster R-CNN — just predicted **all at once**, no region-proposal stage.

Speed vs accuracy — carefully

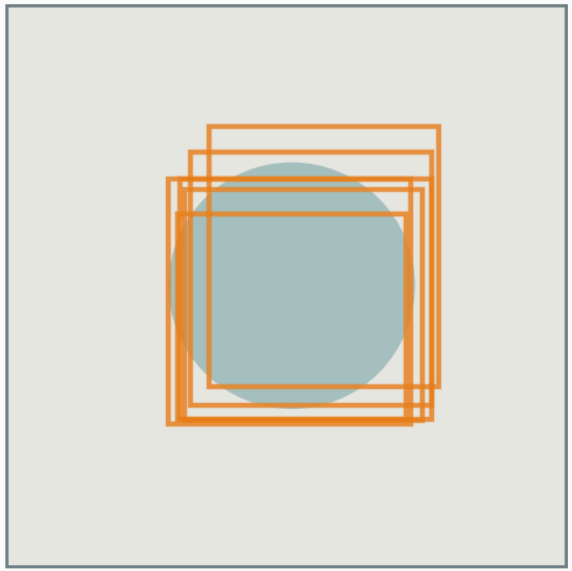
	one-stage (YOLO)	two-stage (Faster R-CNN)
passes	single	propose, then refine
speed	faster, simpler	heavier
localization	historically weaker	historically stronger

These are **historical** tendencies. Modern one-stage detectors (with focal loss, better assignment, stronger backbones) **close much of the gap** — the real ranking depends on implementation, data, and scale.

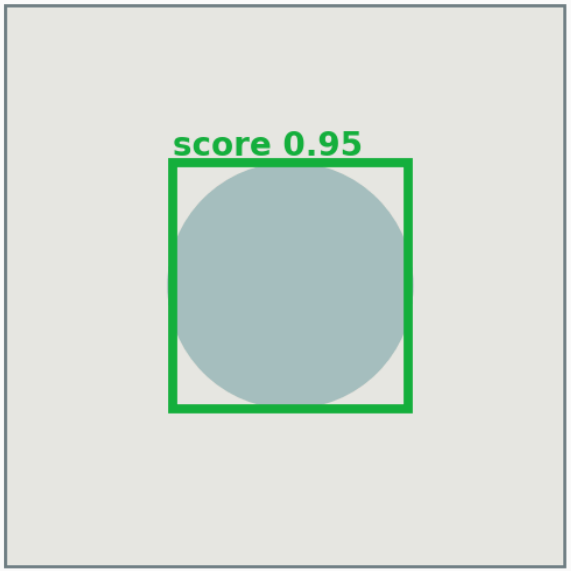
The duplicate problem

The dense grid fires **many** overlapping boxes for one object:

before NMS: many boxes / object



after NMS: keep the best



we want **one** box per object — a post-processing step must **suppress the duplicates**.

Non-Maximum Suppression (NMS)

D

A greedy filter run **per class**, on the raw boxes:

1. **sort** all boxes by confidence score;
2. **keep** the highest-scoring remaining box;
3. **suppress** every other box with $\mathrm{IoU} > \tau$ against it;
4. **repeat** on the boxes that survive.

Each kept box "claims" its neighbourhood; overlapping lower-score boxes are discarded. Typical $\tau \approx 0.5$. (Soft-NMS **decays** scores instead of hard-removing.)

Interactive: NMS

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/object-detection

Start from a pile of overlapping candidate boxes with scores, then step NMS — sort, keep the top box, suppress its high-IoU neighbours — and sweep τ to see over- vs under-suppression.

Q. Two **distinct** objects of the same class stand very close together. What can NMS with a low τ do wrong?

Summary

Tasks and their losses

Task	**Loss**
classification	cross-entropy
localization	cross-entropy $+\lambda$ box
detection	objectness $+$ class $+\lambda$ box

Every step adds a term, never removes one: detection is classification **plus** "is anything here?" **plus** "where exactly?"

The detector families

Detector	Key idea	Stages
R-CNN	region proposals → CNN per crop	two (slow)
Fast R-CNN	shared conv + RoI pooling	two
Faster R-CNN	neural proposals via RPN	two
YOLO	dense prediction in one pass	one

all share: anchors/cells · objectness + class + box · NMS to deduplicate.

Final mental model — one idea

Classification answers **what?**
Detection answers **what + where**,
for a **variable set** of objects.

boxes + scores over dense candidates, cleaned up by NMS

Classification = **what**. Detection = **what + where**, as a set of boxes.

Next (**Lecture 10**): push to **every pixel** — **semantic and instance segmentation**, where the answer is a label map, not a box.